



AUTONOMOUS UNIVERSITY OF THE STATE OF MEXICO

FACULTY OF ENGINEERING

DEGREE IN ARTIFICIAL INTELLIGENCE ENGINEERING

SOFTWARE ENGINEERING II

INTELLIGENT RECOMMENDATION PLATFORM

PROGRAMMER MANUAL

PRESENTED BY:

BETZY PERLA RAMIREZ SOLIS

FABIOLA LAUREANO MARTINEZ

MICHEL GONZALEZ BECERRIL

ANGEL GABRIEL SERRANO VAZQUEZ

SEPTEMBER 21st, 2026.

Fundamental Initial Data

GitHub Repository	https://github.com/Michou001/Intelligent-Recommendation-Plataform.git
Branch / Commit	main / 88f4211
Manual version	1.0
Last Update - Repository	September 19th, 2026
Last Update - Manual	September 21st, 2026
Audience	Developers, reviewers and future maintainers of the project

INTRODUCTION AND PURPOSE

What is this platform?

The **Intelligent Recommendation Platform** is a Python-based web system that suggests movies to users based on their preferences and behaviour. It uses the MovieLens ml-latest-small dataset as its main source of information (around 100,000 user ratings for more than 9,700 movies).

The goal of the platform is to show how a real AI system can be built following good software engineering practices.

How does it work?

It combines multiple artificial intelligence techniques to deliver high-quality recommendations:

- Collaborative Filtering (ALS)**: Analyzes patterns in how users rate movies to find similarities between users with similar tastes.

- Content-Based Filtering**: Examines movie characteristics (genres, descriptions) to recommend films similar to those a user has already liked.

- Deep Learning (NCF)**: Uses Neural Collaborative Filtering, a modern machine learning approach that learns complex relationships between users and movies.

- Popularity Strategy**: A fallback method that recommends the most popular films, especially useful for new users (solving the "cold start" problem).

Who is this manual for?

This developer manual is written for:

- Software engineers who need to understand, maintain, or extend the codebase
- AI/ML practitioners interested in the recommendation algorithms and model architecture
- Students and researchers studying software architecture patterns (Strategy, Factory, Decorator) applied to AI systems
- DevOps engineers responsible for deployment and operation of the platform

What will you learn?

By the end of this manual, you will understand:

- The layered architecture that separates presentation, business logic, AI inference, data access, and persistence
- How to install, configure, and run the system from scratch
- The offline data pipeline that prepares training data
- How the recommendation strategies work and when each one is used
- How to test the system, measure performance, and verify quality
- How to add new features, integrate new models, and scale the architecture

SYSTEM REQUIREMENTS

To successfully install, build, and run the Intelligent Recommendation Platform, your machine must meet the following hardware and software specifications. The system is designed to be lightweight, but specific resources are required for data processing and AI model training.

Hardware Requirements

The platform was benchmarked using a CPU-only environment, meaning a dedicated graphics card (GPU) is not strictly required to run the API or train the base models. However, more resources will improve pipeline execution times.

Component	Minimum Specifications	Recommended Specifications
Processor (CPU)	64-bit processor (x86_64 or ARM64), Dual-core	Quad-core processor or higher (Intel i5 / AMD Ryzen 5 or equivalent)
Memory (RAM)	8 GB (Required for Pandas and PyTorch memory allocation)	16 GB or more (Ideal for handling large Parquet files and caching)
Storage	5 GB of free disk space	10 GB+ SSD (Faster read/write speeds for the SQLite DB and Feature Store)
Network	Broadband connection (Required once to download dependencies and datasets)	Stable internet connection
Graphics (GPU)	Integrated graphics (Sufficient for CPU-only inference)	NVIDIA GPU with CUDA support (Optional, to accelerate Deep Learning / NCF training)

Software Requirements

The platform relies on a modern Python ecosystem. All dependencies are managed automatically via the pyproject.toml configuration file.

1. Operating System

Windows: Windows 10 or 11 (64-bit)

macOS: macOS 11 (Big Sur) or newer

Linux: Ubuntu 20.04 LTS or newer (or any modern Debian/RedHat based distribution)

2. Runtime Environment & Tools

Python: Version 3.11 or higher is mandatory. (The project was developed and verified on CPython 3.13.14).

Package Manager: pip (Included with Python installation).

Version Control: Git (Required to clone the repository).

Terminal: Command Prompt (CMD), PowerShell, Bash, or Zsh.

3. Core Dependencies (Installed automatically via pip)

The system uses several key libraries grouped by their architectural purpose:

- Web Framework: FastAPI and Uvicorn (for the REST API and ASGI server).

- Data Validation: Pydantic v2 (for schema validation and Fail-Fast error handling).

Machine Learning & AI:

- PyTorch (for the Deep Learning / Neural Collaborative Filtering strategy).

- Implicit (for the ALS Collaborative Filtering strategy).

- Scikit-Learn (for Content-Based filtering and cosine similarity).

Data Processing: Pandas and Apache Parquet (for the offline 4-stage data pipeline).

Database: SQLite3 (Built into Python, used for persistence).

4. Development & Quality Assurance Tools

If you plan to modify the code or run the test suite, the following tools are required (also installed via the [dev] tag):

Testing: pytest (Runs the 264 automated unit and integration tests).

Linting & Formatting: ruff, black (Ensures clean code structure).

Static Typing: mypy (Verifies type hints across the codebase).

IDE (Recommended): Visual Studio Code with the official "Python" and "Pylance" extensions installed.

VERIFICATION OF THE PREREQUISITES

Before the project is downloaded, the presence and version of the Python interpreter must be verified. Open a terminal and run the command corresponding to the operating system.

- Windows (PowerShell):

```
python --version
```

- Linux and macOS:

```
python3 --version
```

The command must report version 3.11.0 or later. If it reports an earlier version, or if the command is not recognised, a supported version must be installed from <https://www.python.org/downloads/> before continuing.

Note: On Windows, if the Microsoft Store opens instead of a version being reported, the Store execution alias is intercepting the command. Either install Python from python.org with the option **Add python.exe to PATH** selected, or disable the alias under Settings, Apps, Advanced app settings, App execution aliases. The launcher `py -3.11 --version` may also be used as an alternative.

The availability of pip and of the venv module must also be confirmed.

- Windows:

```
python -m pip --version
```

```
python -m venv --help
```

- Linux and macOS:

```
python3 -m pip --version
```

```
python3 -m venv --help
```

If either command reports that the module cannot be found on a Debian- or Ubuntu-based distribution, the corresponding system packages must be installed:

```
sudo apt install python3-venv python3-pip
```

On macOS, the command line developer tools must be available; if they are not, they are installed with:

```
xcode-select --install
```

Obtaining the Project

The project may be obtained in either of two ways: by downloading the repository as a compressed file, or by cloning it with Git. Only one of the two procedures is required.

Option A. Downloading and extracting the compressed file

Once the folder has been downloaded from the GitHub repository Intelligent-Recommendation-Platform, the location in which the compressed file has been saved must be identified, since that path is required in the following steps. The file is then extracted. The process shown below corresponds to the Windows operating system.

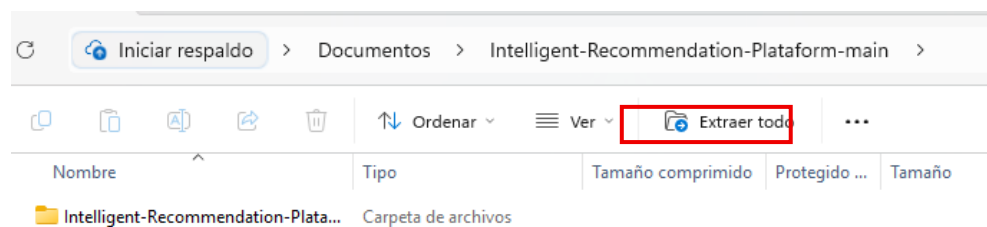


Figure 1. Location of the downloaded compressed file in File Explorer (Windows).

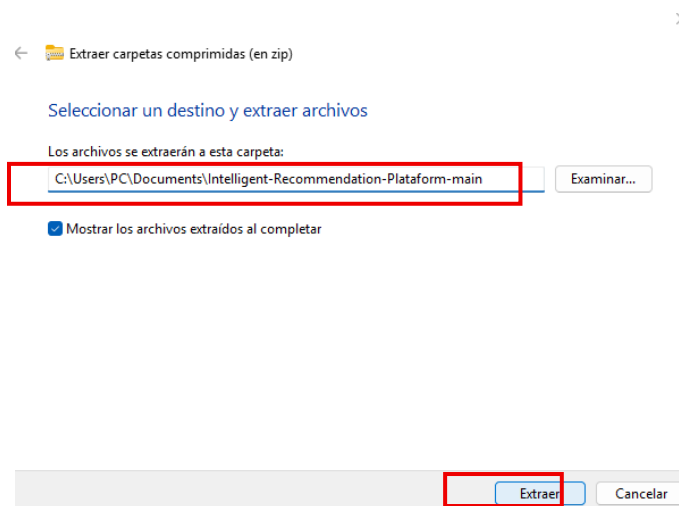


Figure 2. Extraction of the compressed file (Windows).

The equivalent commands for each operating system are the following. They must be run in the directory that contains the downloaded file.

- Windows (PowerShell):

Expand-Archive -Path .Intelligent-Recommendation-Platform-main.zip -DestinationPath

- Linux and macOS:

unzip Intelligent-Recommendation-Platform-main.zip

Note. If the `unzip` utility is not installed on Linux, it can be added with `sudo apt install unzip`. On macOS, the compressed file may also be extracted by double-clicking it in Finder, and on Windows by using the Extract All option of the context menu.

The directory in which the folder is saved must be noted, because that address has to be entered in the terminal in Section 5.

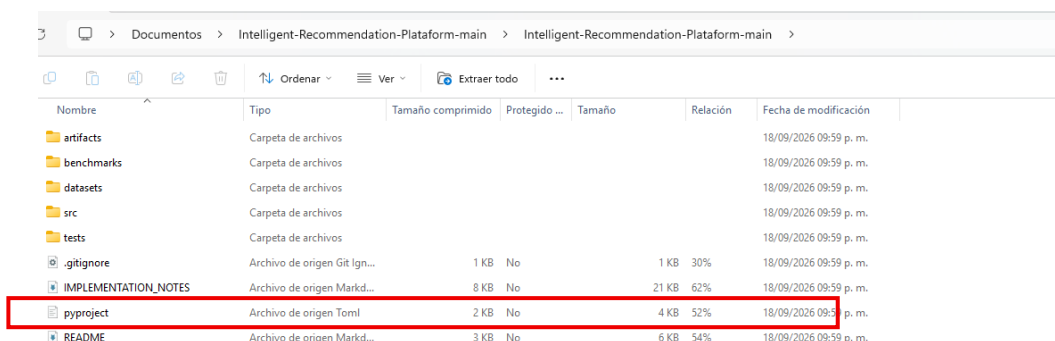
Option B. Cloning the repository with Git

If Git is installed, the project may be obtained with a single command, which is identical on Windows, Linux and macOS. In this case no extraction step is necessary.

```
git clone https://github.com/<ORGANISATION>/Intelligent-Recommendation-Platform.git
cd Intelligent-Recommendation-Platform
```

Opening a Terminal in the Project Directory

A terminal must be opened and the working directory changed to the root of the project, that is, the directory that contains the file `pyproject.toml`.



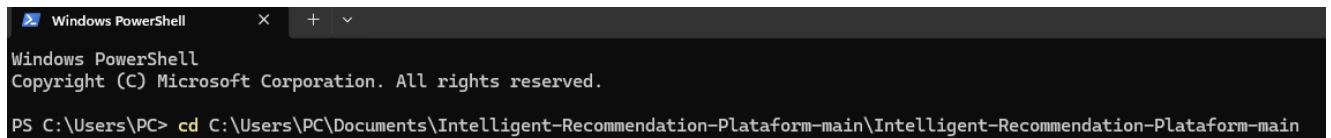
Nombre	Tipo	Tamaño comprimido	Protegido ...	Tamaño	Relación	Fecha de modificación
artifacts	Carpeta de archivos					18/09/2026 09:59 p. m.
benchmarks	Carpeta de archivos					18/09/2026 09:59 p. m.
datasets	Carpeta de archivos					18/09/2026 09:59 p. m.
src	Carpeta de archivos					18/09/2026 09:59 p. m.
tests	Carpeta de archivos					18/09/2026 09:59 p. m.
.gitignore	Archivo de origen Git Ign...	1 KB	No	1 KB	30%	18/09/2026 09:59 p. m.
IMPLEMENTATION_NOTES	Archivo de origen Markd...	8 KB	No	21 KB	62%	18/09/2026 09:59 p. m.
pyproject	Archivo de origen Toml	2 KB	No	4 KB	52%	18/09/2026 09:59 p. m.
README	Archivo de origen Markd...	3 KB	No	6 KB	54%	18/09/2026 09:59 p. m.

Figure 3. Contents of the project folder, including the file `pyproject.toml` (Windows).

The command used to change directory is given below for each operating system. The value in angle brackets must be replaced with the actual path of the project.

- Windows (PowerShell or Command Prompt):
`cd "C:\<PATH-TO-THE-PROJECT>"`
- Linux:
`cd "/home/<USER>/<PATH-TO-THE-PROJECT>"`
- macOS:
`cd "/Users/<USER>/<PATH-TO-THE-PROJECT>"`

Note. Quotation marks are strictly necessary only when the path contains spaces, but their use is recommended in every case. On Linux and macOS, paths are case-sensitive and use a forward slash as separator; on Windows the separator is a backslash.



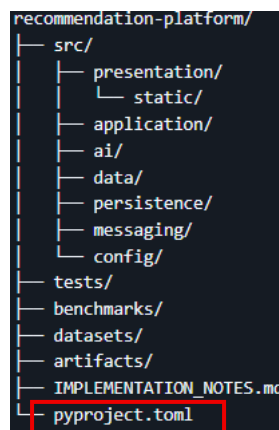
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\PC> cd C:\Users\PC\Documents\Intelligent-Recommendation-Platform-main\Intelligent-Recommendation-Platform-main
```

Figure 4. Result of the change of directory in the terminal.

Before continuing, it must be verified that the current directory contains the file `pyproject.toml`, which is located in the root directory of the project.

- Windows (PowerShell):
`dir pyproject.toml`
- Linux and macOS:
`ls pyproject.toml`



```
recommendation-platform/
├── src/
│   ├── presentation/
│   │   └── static/
│   ├── application/
│   ├── ai/
│   ├── data/
│   ├── persistence/
│   ├── messaging/
│   └── config/
├── tests/
├── benchmarks/
├── datasets/
├── artifacts/
├── IMPLEMENTATION_NOTES.md
└── pyproject.toml
```

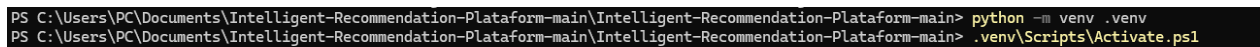
Figure 5. Expected directory structure of the project.

If the file is not listed, the current directory is not the root of the project and the change of directory must be repeated with the correct path.

Creating and Activating the Virtual Environment

Inside the project directory, the following commands are run in order to create and activate the virtual environment. The virtual environment isolates the dependencies of the project from the Python installation of the operating system.

- Windows (PowerShell):
`python -m venv .venv`
`.\.venv\Scripts\Activate.ps1`
- Windows (Command Prompt, cmd.exe):
`python -m venv .venv`
`.venv\Scripts\activate.bat`
- Linux and macOS:
`python3 -m venv .venv`
`source .venv/bin/activate`



```
PS C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main> python -m venv .venv
PS C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main> .venv\Scripts\Activate.ps1
```

Figure 6. Creation and activation of the virtual environment.

Note. If PowerShell reports that the file `Activate.ps1` cannot be loaded because the running of scripts is disabled on the system, the execution policy must be relaxed for the current session only, and the activation command repeated:

`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Successful activation is confirmed by the prefix `(.venv)` shown at the beginning of the command prompt. The virtual environment must remain active throughout the remaining sections; if the terminal is closed, the activation command has to be run again before continuing.

Installing the Project and Its Dependencies

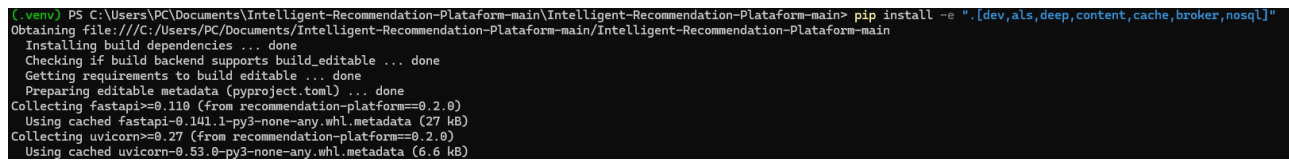
With the virtual environment active, the packaging tools are first brought up to date.

- Windows:
`python -m pip install --upgrade pip setuptools wheel`

- Linux and macOS:
`python3 -m pip install --upgrade pip setuptools wheel`

The project and its optional dependency groups are then installed in editable mode. The following command is identical on the three operating systems.

```
pip install -e "[dev,als,deep,content,cache,broker,nosql]"
```



```
[...venv] PS C:\Users\PC\Documents\Intelligent-Recommendation-Platform-main\Intelligent-Recommendation-Platform-main> pip install -e "[dev,als,deep,content,cache,broker,nosql]"
Obtaining file:///C:/Users/PC/Documents/Intelligent-Recommendation-Platform-main/Intelligent-Recommendation-Platform-main
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting fastapi==0.110 (from recommendation-platform==0.2.0)
Using cached fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn==0.27 (from recommendation-platform==0.2.0)
Using cached uvicorn-0.53.0-py3-none-any.whl.metadata (6.6 kB)
```

Figure 7. Installation of the project and its dependencies.

Note. The double quotation marks are mandatory. Without them, Bash and Zsh interpret the square brackets as a file-name pattern and the command fails with a message such as no matches found. The installation may take several minutes, since a large number of packages are downloaded; completion is indicated by a message beginning with Successfully installed.

Running the Automated Tests

The following command runs the automated test suite of the project.

- Windows:
`python -m pytest`
- Linux and macOS:
`python3 -m pytest`

```

(.venv) PS C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main> python -m pytest
..... [ 76%]
..... [100%]
===== warnings summary =====
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main\venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'htpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
    from starlette.testclient import TestClient as TestClient # noqa

.venv\Lib\site-packages\starlette\testclient.py:53
  C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main\venv\Lib\site-packages\starlette\testclient.py:53: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

tests\data\pipeline\test_pipeline.py::test_transformation_produces_the_four_documented_outputs
  C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main\venv\Lib\site-packages\implicit\cpu\als.py:96: RuntimeWarning: OpenBLAS is configured to use 16 threads. It is highly recommended to disable its internal threadpool by setting the environment variable 'OPENBLAS_NUM_THREADS=1' or by calling 'threadpoolctl.threadpool_limits(1, "blas")'. Having OpenBLAS use a threadpool can lead to severe performance issues here.
    check_blas_config()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
264 passed, 3 warnings in 22.49s

```

Figure 8. Result of the execution of the automated tests.

The execution is considered successful when every test is reported as passed and no failures or errors are listed in the summary. If any test fails, the installation described in Section 7 must be verified before continuing.

Static Analysis and Architecture Import Verification

The following commands perform the static analysis of the source code and the strict verification of the architectural import rules.

- Windows:
 - `python -m ruff check`
 - `python -m mypy`
- Linux and macOS:
 - `python3 -m ruff check`
 - `python3 -m mypy`

```

(.venv) PS C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main> python -m ruff check .
All checks passed!
(.venv) PS C:\Users\PC\Documents\Intelligent-Recommendation-Plataform-main\Intelligent-Recommendation-Plataform-main> python -m mypy
Success: no issues found in 39 source files

```

Figure 9. Result of the static analysis and of the type verification.

Note. On Linux and macOS, command names are case-sensitive. The interpreter must be written as `python3`, entirely in lower case; the form `Python3` is not recognised and produces a command not found error.

Latency Benchmark

The latency benchmark measures the response time of the system and is run in two configurations, with and without cache, so that the effect of the cache can be compared.

- Test with cache, Windows:
`python benchmarks/latency_p95.py`
- Test with cache, Linux and macOS:
`python3 benchmarks/latency_p95.py`

```
Latency benchmark - NFR-01 target: 200 ms at p95
-----
mode                controller
strategy            als
caching             True
k                   10
dataset_variant      ml-latest-small
feature_store_version v1
requests            500
errors              0
degraded_responses  0
min_ms              0.392
mean_ms             0.626
p50_ms              0.617
p95_ms              0.888
p99_ms              1.051
max_ms              2.164
target_p95_ms        200.0
headroom_ms          199.112
verdict              PASS
-----
VERDICT: PASS
```

Figure 10. Result of the latency benchmark with cache.

- Test without cache, Windows:
`python benchmarks/latency_p95.py --no-cache`
- Test without cache, Linux and macOS:
`python3 benchmarks/latency_p95.py --no-cache`

```
Latency benchmark - NFR-01 target: 200 ms at p95
-----
mode                controller
strategy            als
caching             False
k                   10
dataset_variant      ml-latest-small
feature_store_version v1
requests            500
errors              0
degraded_responses  0
min_ms              0.493
mean_ms             0.651
p50_ms              0.632
p95_ms              0.826
p99_ms              0.955
max_ms              1.225
target_p95_ms        200.0
headroom_ms          199.174
verdict              PASS
-----
VERDICT: PASS
```

Figure 11. Result of the latency benchmark without cache.

Note. The forward slash used in the path of the script is accepted by the Python interpreter on Windows as well, so the command does not have to be modified.

SYSTEM ARQUITECTURE

The platform follows a **Layered Architecture** design that respects **SOLID principles** and uses **Gang of Four (GoF) design patterns** to ensure maintainability, scalability, and separation of concerns.

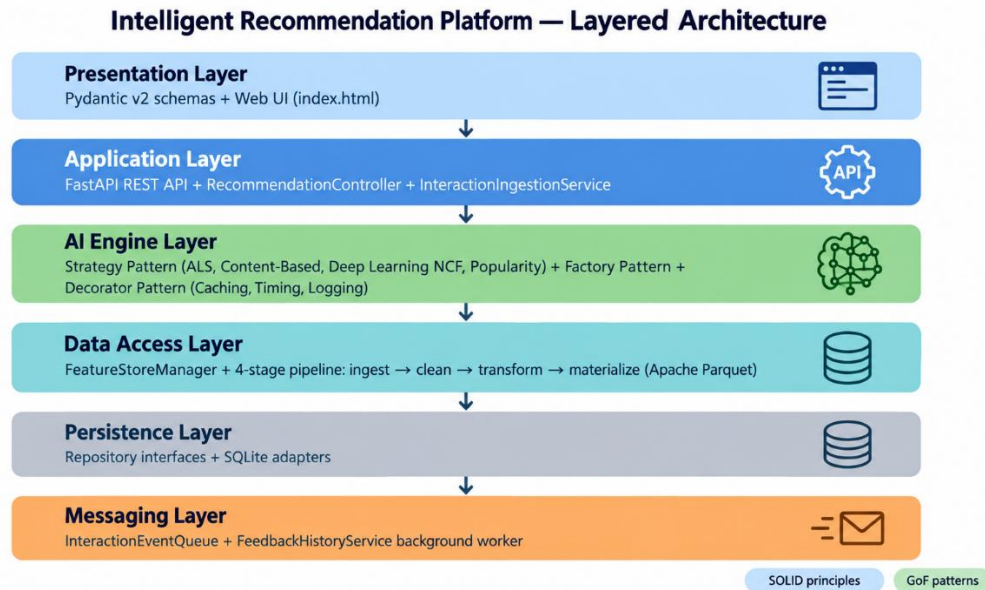


Figure 1. Layered architecture diagram of the platform

LAYERS' EXPLANATION

Layer 1: Presentation Layer (src/presentation/)

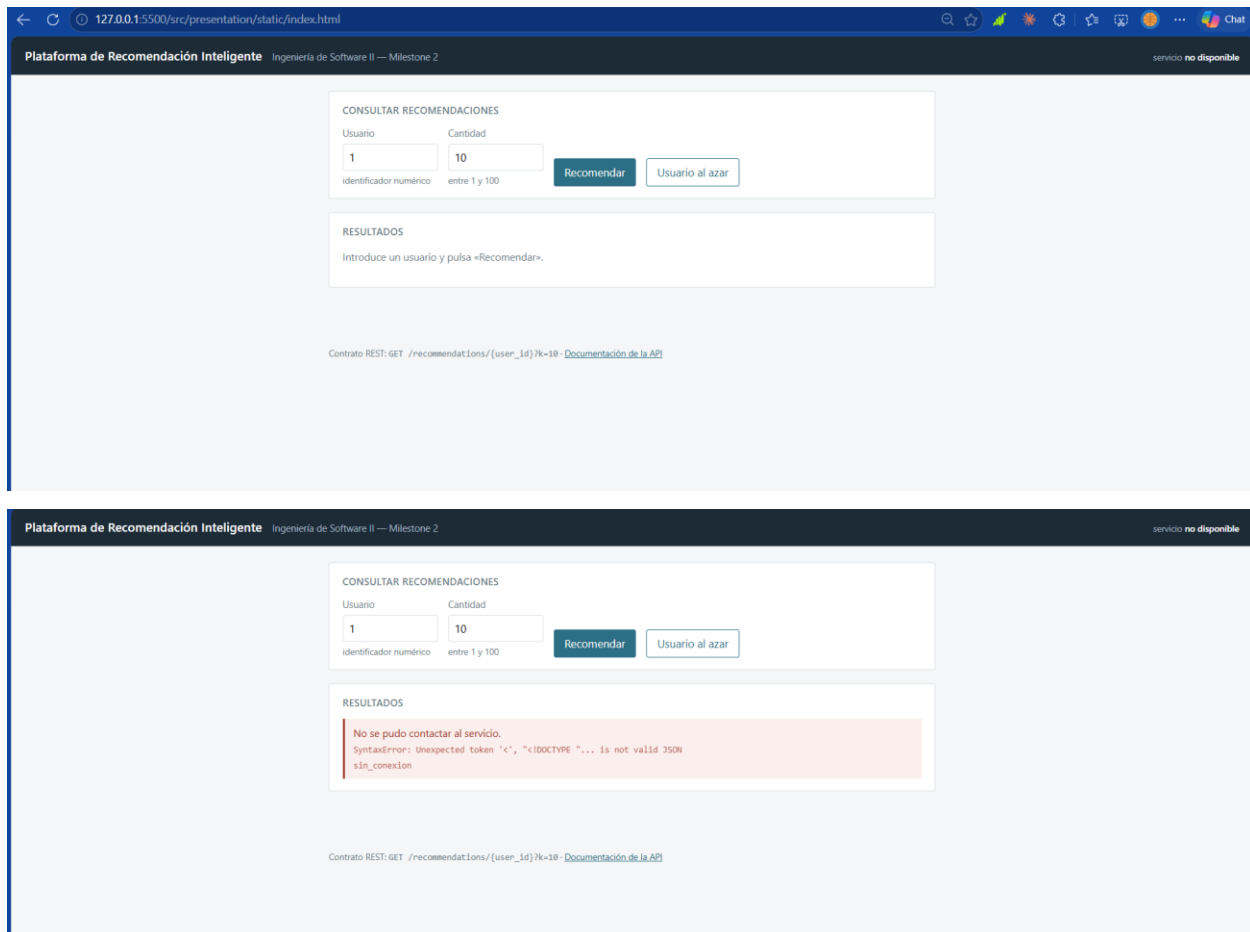
-Purpose: Validates incoming requests and provides the user interface

-Components:

- Pydantic v2 schemas for request/response validation (Fail-Fast approach)

- Static web interface (index.html) that displays recommendations

Key Feature: Returns HTTP 400 for invalid input before any processing



Layer 2: Application Layer (src/application/)

-Purpose: Orchestrates the entire recommendation flow

-Components:

- RecommendationController: Coordinates strategy selection and execution

- InteractionIngestionService: Captures real-time user interactions

- FastAPI REST endpoints for external communication

Key Feature: Decouples HTTP logic from AI inference

Layer 3: AI and Recommendation Engine Layer (src/ai/)

-Purpose: Contains the core intelligence of the system

-Design Patterns Used:

- Strategy Pattern: Interchangeable recommendation models implementing `IRecommendationStrategy.predict()`

- Factory Pattern: Registry-based model instantiation with `@register_strategy` decorator

- Decorator Pattern: Dynamic operational enhancements

-Components:

- CollaborativeFilteringStrategy (implicit ALS)

- ContentBasedStrategy (cosine similarity)

- DeepLearningStrategy (PyTorch NCF)

- PopularityStrategy (cold-start fallback)

- CachingDecorator, TimingDecorator, LoggingDecorator

Layer 4: Data Access and Feature Store Layer (src/data/)

-Purpose: Manages precomputed features and serves them efficiently

-Components:

- FeatureStoreManager: Reads Apache Parquet files containing user/item vectors

- 4-Stage Offline Pipeline: ingest → clean → transform → materialize

-Key Feature: Precomputes features offline for fast online inference

Layer 5: Persistence and Storage Layer (src/persistence/)

-Purpose: Stores and retrieves persistent data using the Repository pattern

-Components:

- Segregated repository interfaces (IUserProfileRepository, ICatalogRepository, etc.)

- SQLite adapters for local storage

-Key Feature: Interface segregation allows easy database swapping

Layer 6: Messaging and Asynchronous Logging (src/messaging/)

-Purpose: Handles background tasks without blocking the main request path

-Components:

- InteractionEventQueue: Queues user interactions for asynchronous processing

- FeedbackHistoryService: Background worker that consumes events

Key Feature: Improves system responsiveness and reliability

This architecture ensures that the AI inference engine can evolve independently from business logic and persistence layers, making the system highly maintainable and extensible.

PIPELINE DATA PROCESS

1. Train the Deep Learning model

```
python -m ai.strategies.deep_learning_strategy --train
```

What it does: Trains the Neural Collaborative Filtering (NCF) model using PyTorch.

Expected result: The terminal shows the training loss decreasing epoch by epoch (for example, from 0.4129 to 0.3486 over 3 epochs) and saves the model weights into the artifacts/ folder.

2. Seed the database

```
python -m persistence --seed
```

What it does: Loads the user profiles, the movie catalog, and the item embeddings into the SQLite database.

Expected result: A confirmation message indicating that profiles, catalog, and embeddings were seeded successfully.

ACTIVATION PROCESS

Once we've installed the repository in our computers, we can use it following these steps:

NOTE: all these commands we're applied to in CMD, so you've got to adapt all of them to your command's execution platform.

1. Go to the direction folder

```
cd "C:\Users\angel\Desktop\Intelligent-Recommendation-Plataform"
```

```
C:\Users\angel>cd "C:\Users\angel\Desktop\Intelligent-Recommendation-Plataform"
```

2. Activate your virtual environment

```
.venv\Scripts\activate
```

```
C:\Users\angel\Desktop\Intelligent-Recommendation-Plataform>.venv\Scripts\activate
```

3. Initialize the server

```
set PYTHONPATH=src
```

```
(.venv) C:\Users\angel\Desktop\Intelligent-Recommendation-Plataform>set PYTHONPATH=src
```

Note: this command isn't going to show anything in the command prompt

```
python -m uvicorn application.api:app --reload --port 8000
```

```
(.venv) C:\Users\angel\Desktop\Intelligent-Recommendation-Plataform>python -m uvicorn application.api:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\angel\\Desktop\\Intelligent-Recommendation-Plataform']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [4976] using StatReload
INFO: Started server process [17192]
INFO: Waiting for application startup.
INFO: Application startup complete.
{"backend": "in-memory", "event": "worker.started", "level": "info", "timestamp": "2026-09-21T01:11:41.479277Z"}
```

With this last command we have everything activated, so we can interact with the UI in this port in our browser:

<http://127.0.0.1:8000/ui/>

The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/ui/`. The page title is "Plataforma de Recomendación Inteligente" and the subtitle is "Ingeniería de Software II — Milestone 2". The top right corner shows "servicio operativo · modelo als · catálogo 7.343".

The main content area is divided into two sections:

- CONSULTAR RECOMENDACIONES**: This section contains two input fields: "Usuario" with the value "7" (range "de 1 a 609") and "Cantidad" with the value "10" (range "entre 1 y 100"). There are two buttons: "Recomendar" (highlighted in green) and "Usuario al azar".
- RESULTADOS**: This section displays a list of 8 movie recommendations. Each entry includes a rank, the movie title, genres, a score, and a "Me interesa" button.

Rank	Movie Title	Genres	Score	Action
1	Finding Nemo (2003)	Adventure · Animation · Children · Comedy	0.889	Me interesa
2	Godfather, The (1972)	Crime · Drama	0.767	Me interesa
3	Legally Blonde (2001)	Comedy · Romance	0.767	Me interesa
4	Bourne Identity, The (2002)	Action · Mystery · Thriller	0.752	Me interesa
5	Speed (1994)	Action · Romance · Thriller	0.749	Me interesa
6	Pulp Fiction (1994)	Comedy · Crime · Drama · Thriller	0.743	Me interesa
7	Shawshank Redemption, The (1994)	Crime · Drama	0.741	Me interesa
8	Fifth Element, The (1997)		0.690	Me interesa

UI ERROR HANDLING (Examples)

The screenshot shows a web browser window with the URL `127.0.0.1:8000/ui/`. The page title is "Plataforma de Recomendación Inteligente" and the subtitle is "Ingeniería de Software II — Milestone 2". The top right corner shows "servicio operativo · modelo ale · catálogo 7.343". The main content area has a header "CONSULTAR RECOMENDACIONES" with two input fields: "Usuario" (containing "7000") and "Cantidad" (containing "100"). Below the "Usuario" field is a range indicator "de 1 a 609". Below the "Cantidad" field is a range indicator "entre 1 y 100". There are two buttons: "Recomendar" and "Usuario al azar". Below the inputs is a section titled "RESULTADOS" which contains a red error message: "user 7000 does not exist" and "user_not_found".

A higher user number that overpasses the allowed range

The screenshot shows the same web application. The "Usuario" field contains "7" and the "Cantidad" field contains "99999". Below the "Cantidad" field is a range indicator "entre 1 y 100". A red error message is displayed below the "Recomendar" button: "Value must be less than or equal to 100.".

A higher recommendation number that overpasses the allowed range

The screenshot shows the same web application. The "Usuario" field contains "258" and the "Cantidad" field contains "99999". Below the "Cantidad" field is a range indicator "entre 1 y 100". The "RESULTADOS" section contains a red error message: "the request does not satisfy the declared schema" and "query.k: Input should be less than or equal to 100" and "validation_error".

A higher recommendation number that overpasses the allowed range (random user case)

The screenshot shows the same web application. The "Usuario" field contains "1" and the "Cantidad" field contains "10". Below the "Usuario" field is a range indicator "de 1 a 609". Below the "Cantidad" field is a range indicator "entre 1 y 100". The "RESULTADOS" section contains a red error message: "No se pudo contactar al servicio." and "SyntaxError: Unexpected token '<', '<DOCTYPE' ... is not valid JSON" and "sin_conexion". At the bottom of the page, there is a footer: "Contrato REST: GET /recommendations/{user_id}?k=10 · Documentación de la API".

Connection Error with the server.